

PERT-диаграмма проекта и программной системы

Описание проекта

Программный продукт **автоматизирует процесс установки и настройки операционной системы**, а именно позволяет:

- Конфигурировать модули ядра через *YAML*-файл
- Принимать решения касательно *system derivation* при неполном описании системы

Ключевые функции:

- Генерация конфигурации ядра на основе *YAML*-описания
- Управление пакетами с поддержкой *USE*-флагов
- Автоматическая установка необходимых модулей
- Пост-установочная конфигурация служб

Работа нацелена на:

- Пользователей, желающих легко разворачивать воспроизводимые системы, а именно:
 - Администраторов *Linux*-систем
 - *DevOps*-специалистов
 - Энтузиастов, ценящих декларативный подход к конфигурированию

Задание:

1. Представьте индивидуальный программный проект с помощью PERT – диаграммы с точки зрения управления проектом.

2. Представьте индивидуальный программный проект с помощью PERT – диаграммы с точки программной системы.

3. Ответьте письменно на следующие вопросы:

- Дайте определение программной спецификации, для чего она нужна;
- Перечислите основные свойства спецификаций;
- Перечислите основные характеристики языков спецификаций
- Опишите методы структурного анализа;
- Дайте определение PERT-диаграммы, для каких целей они применяются.
- Выводы.

PERT-диаграмма проекта (управление проектом)

Программный продукт автоматизирует процесс установки и настройки операционной системы, а именно позволяет:

- Конфигурировать модули ядра через *YAML*-файл
- Принимать решения касательно *system derivation* при неполном описании системы

Набор работ:

T1 – Анализ требований к автоматизации развёртывания ОС (1 неделя)

T2 – Проектирование архитектуры системы (модули генерации конфигурации, управления пакетами, пост-установки) (2 недели)

T3 – Разработка генератора конфигурации ядра на основе *YAML*-описания (3 недели)

T4 – Реализация модуля управления пакетами с поддержкой USE-флагов (2 недели)

T5 – Создание системы автоматической установки модулей ядра (2 недели)

T6 – Разработка механизма пост-установочной конфигурации служб (2 недели)

T7 – Интеграция модулей и внутреннее тестирование (функциональное, интеграционное) (2 недели)

T8 – Подготовка документации и примеров использования (1 неделя)

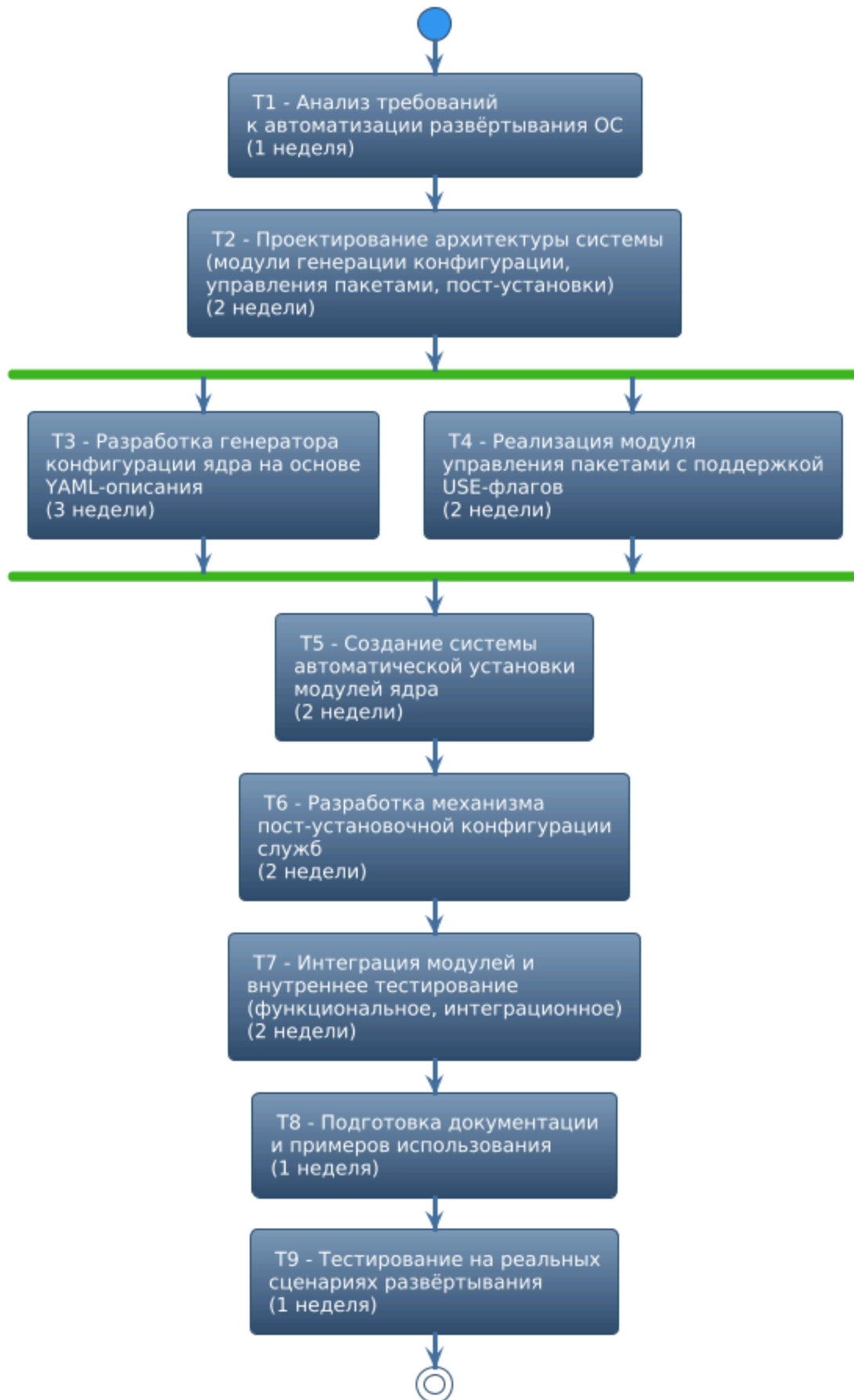
T9 – Тестирование на реальных сценариях развёртывания (1 неделя)

Зависимости:

T1 → T2 → (T3 и T4 параллельно) → T5 → T6 → T7 → T8 → T9.

Критический путь: T1–T2–T3–T5–T6–T7–T9 (суммарная длительность: 11 недель).

PERT-диаграмма проекта (управление проектом)



```

@startuml
!theme spacelab
title PERT-диаграмма проекта (управление проектом)

start
: T1 - Анализ требований\пк автоматизации развёртывания ОС\n(1 неделя);
: T2 - Проектирование архитектуры системы\n(модули генерации
конфигурации,\пуправления пакетами, пост-установки)\n(2 недели);

fork
: T3 - Разработка генератора\пконфигурации ядра на основе\nYAML-описания\n(3
недели);
fork again
: T4 - Реализация модуля\пуправления пакетами с поддержкой\nUSE-флагов\n(2
недели);
end fork

: T5 - Создание системы\павтоматической установки\nмодулей ядра\n(2 недели);
: T6 - Разработка механизма\ппост-установочной конфигурации\nслужб\n(2 недели);
: T7 - Интеграция модулей и \пвнутреннее тестирование\n(функциональное,
интеграционное)\n(2 недели);
: T8 - Подготовка документации\пи примеров использования\n(1 неделя);
: T9 - Тестирование на реальных\псценариях развёртывания\n(1 неделя);

stop
@enduml

```

PERT-диаграмма программной системы (уровень модулей)

- Событие E0 – Запуск системы автоматизации развёртывания.
 - Работа A1: Чтение и валидация YAML-конфигурации (загрузка параметров ядра, пакетов, служб).
- Событие E1 – Конфигурация успешно загружена.
 - Работа A2: Генерация конфигурации ядра и подготовка к компиляции.
- Событие E2 – Ядро сконфигурировано.
 - Работа A3: Установка пакетов с учётом USE-флагов и зависимостей.
- Событие E3 – Пакеты установлены.
 - Работа A4: Автоматическая загрузка и активация модулей ядра.
- Событие E4 – Модули ядра активированы.
 - Работа A5: Настройка служб (сетевые демонов, планировщики задач и др.).
- Событие E5 – Система готова к использованию.

В PERT-диаграмме модули (A1–A5) изображаются стрелками, события (E0–E5) – вершинами, отражая последовательность активации компонентов системы.



```
@startuml
!theme cerulean
title PERT-диаграмма программной системы\n(уровень модулей)

left to right direction

' Определение событий (вершины)
circle E0 as "E0"
circle E1 as "E1"
circle E2 as "E2"
circle E3 as "E3"
circle E4 as "E4"
circle E5 as "E5"

' Определение работ (стрелки между событиями)
E0 --> E1 : A1\nЧтение и валидация\nYAML-конфигурации\n(загрузка параметров\nядра,\nпакетов, служб)
E1 --> E2 : A2\nГенерация конфигурации\nядра и подготовка\nк компиляции
E2 --> E3 : A3\nУстановка пакетов\nс учётом USE-флагов\nи зависимостей
E3 --> E4 : A4\nАвтоматическая загрузка\nи активация модулей\nядра
E4 --> E5 : A5\nНастройка служб\n(сетевые демоны,\nпланировщики задач и др.)

note top of E5
  <b>Конечное событие</b>
  Система полностью настроена и готова к эксплуатации
end note
@enduml
```